

Locking Secret Data in the Vault Leveraging Fuzzy PUFs

Paul Chin, Yuan Cao*, Xiaojin Zhao[†], Leilei Zhang[‡], and Fan Zhang^{§¶}

**Hohai University, Chang Zhou, China*

[†]College of Electronics and Information Engineering, Shen Zhen University, Shen Zhen, China

[‡]Fiberhome Telecommunication Technologies Co. Ltd, Wu Han, China

[§]School of Cyber Science & Technology, College of computer Science and Technology, Zhejiang University, Hang Zhou, China

[¶]College of Information Science & Electronic Engineering, Zhejiang University, Hang Zhou, China

Corresponding Author: Fan Zhang, email: fanzhang@zju.edu.cn

Abstract—Physical Unclonable Functions (PUFs) are considered as an attractive low-cost security anchor. The unique features of PUFs are dependent on the Nanoscale variations introduced during the manufacturing variations. Most PUFs exhibit an unreliability problem due to aging and inherent sensitivity to the environmental conditions. As a remedy to the reliability issue, helper data algorithms are used in practice. A helper data algorithm generates and stores the helper data in the enrollment phase in a secure environment. The generated helper data are used then for error correction, which can transform the unique feature of PUFs into a reproducible key. The key can be used to encrypt secret data in the security scheme. In contrast, this work shows that the fuzzy PUFs can be used to secret important data directly by an error-tolerant protocol without the enrollment phase and error-correction algorithm. In our proposal, the secret data is locked in a vault leveraging the unique fuzzy pattern of PUF. Although the noise exists, the data can then be released only by this unique PUF. The evaluation was performed on the most prominent intrinsic PUF - DRAM PUF. The test results demonstrate that our proposal can reach an acceptable reconstruction rate in various environment. Finally, the security analysis of the new proposal is discussed.

Index Terms—Physical unclonable functions, error-tolerant, fuzzy pattern.

1. Introduction

Physical Unclonable Functions (PUFs) leverage the unique feature of the device as a hardware fingerprint. The unique feature of PUF are dependent on the nanoscale variations of device introduced during the manufacturing process, which is difficult to be replicated. Therefore, PUFs are considered as an attractive low-cost security anchor to mitigate some potential pitfalls in Non-Volatile Memory (NVM)-based security scheme.

As a promising cryptographic primitive in key storage, authentication, and identification scheme, it is extremely important that the PUFs function correctly during the lifetime

of the devices. However, most PUFs exhibit an unreliability problem due to aging and inherent sensitivity to the environmental conditions. As a remedy to the reliability issue, helper data algorithms [1] are used in practice, which can transform the unique feature of PUFs into a reproducible high-entropy key. Furthermore, the privacy algorithm (e.g., hash algorithm) is used to maximise the entropy of key. This hardware-based unique key can be used to encrypt the important data, e.g., the application key [2] [3]. However, paper [1] mentioned that most work on helper data algorithms considers the "reproducibility" and "uniformly distributed" only and ignored the key requirements of "PUFs independency", "mathematical restrictions" and "full control".

Juel and Sudan's Fuzzy Vault [4] is an error-tolerant cryptographic construction. It is an effective way for biometric template protection in biometric identification. In the Fuzzy Vault algorithm, the fuzzy biometric data can be hidden with secret information in randomly selected chaff points to ensure data security. Although the biometric data suffers from the noise effects, the feature of error-tolerant of Fuzzy Vault allows the identification success. The error-tolerant feature of Fuzzy Vault is suitable for PUFs. In this work, we show that the secret data can be locked in vault leveraging the fuzzy PUFs. In our proposal, the secret data can be unlocked when part of the PUF features remain stable. Our key contributions are as follows:

- We demonstrate that the fuzzy PUFs can be used to secret store the important data without the error-correction algorithm, which is a new train of thought about how to use PUFs in an embedded system.
- Evaluation of the new proposal with DRAM PUFs in the off-the-shelf commodity devices without any hardware modifications.
- Discussion of the security and possible improvements of the new proposal.

2. Background

2.1. Helper Data Algorithm

To use PUFs as reproducible key or identification, helper data algorithm, also known as Fuzzy Extractor [5] is required. The helper data algorithm consists of an enrollment phase and a reconstruction phase. In the enrollment phase, the helper data can be generated by $h = GEN(r)$, where GEN is the enrollment process of helper data algorithm and r is the raw PUF measurement. Helper data h can be stored publicly as it should not leak any information about r . At the reconstruction phase, a fresh measurement of PUF r' is queried from the target device. Using the helper data algorithm, r can be regenerated by $r = REGEN(r', h)$, where $REGEN$ is the error correction process of helper data algorithm if the Hamming Distance of r and r' is smaller than the error-correcting capability of the help data algorithm.

To avoid leakage problems due to the bias of DRAM PUF, paper [6] proposed a two-layer Helper Data Algorithm. The first layer selects the cells with higher reliability to improve noise resilience. The second layer uses the pointers which identify a similar number of stable fast cells and slow cells to avoid the bias problem.

2.2. DRAM PUF

As shown in Figure 1, each DRAM cell consists of a transistor and a capacitor. The DRAM cell stores one-bit data as the charge on a capacitor. The cells are placed in a 2-dimensional array, where each row of the array is connected to a word line in horizontal. In the column, cells are linked by a bit line in vertical. Amplifiers and equalizers are used to sense the small voltage differences and amplifies it to voltage levels corresponding to logical ones or zeroes. In DRAM, the capacitors leak charge over time (some of the leakage paths are shown in Figure 1), which may result in loss of data stored in the DRAM cell. Therefore, DRAM chips usually have a periodic self-refresh, which controlled by the memory controller to replenish the charge before the bits can flip.

Prior works have shown that DRAM can be used as an attractive, low-cost intrinsic PUFs in commercial off-the-shelf devices, which do not require any hardware modifications [7]. Three types of DRAM PUFs have been proposed based on: the decay feature of DRAM cells [7], startup values [8] of the DRAM cells, and the write latency [9] of the DRAM cell. To the best knowledge of the authors, in all the DRAM PUF implementations, decay-based DRAM PUFs are the only DRAM PUF implementation that is accessible at the run- time of the device.

The so-called Retention Time is the amount of time that the DRAM cell can reliably hold data. Retention time is sensitive to the manufacturing variations. If the self-refresh is disabled, Some cells leak faster than others. After a certain decay time, if enough charge has leaked, the cell will flip their value if they were previously in a charged state. But

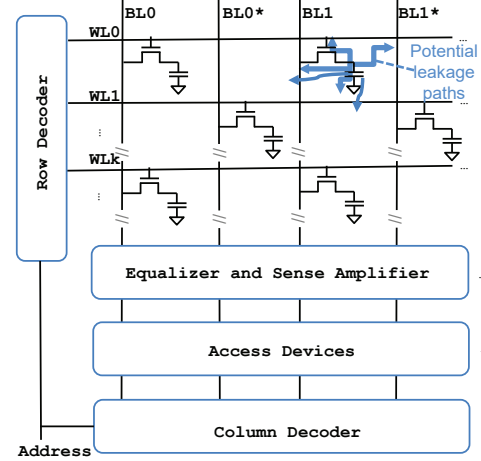


Figure 1. The architecture of DRAM.

for the other cells, the contents will stay stable. Due to the manufacturing variations, the decay feature of DRAM has been shown to be a good candidate for using as PUFs.

3. Protecting the Secret Data by Fuzzy PUFs

We present our *LOCK* algorithm in Algorithm 1 and our *UNLOCK* algorithm in Algorithm 2

In *LOCK* process, the secret message M is in the finite field $GF(2^{16})$. an error detecting code is chosen that encodes k -bit message M in n -bit codewords S , e.g., Cyclic Redundancy Check (CRC) [10]. Based on the codewords S , a t -th-order polynomial P can be constructed by partitioning S into $t+1$ parts $S_0 \dots S_t$ and considering them as the coefficients of polynomial $P(x) = S_t \cdot x^t + S_{t-1} \cdot x^{t-1} + \dots + S_0$. The polynomial P contains all the secret message. It will be protected by the Fuzzy PUF.

The system queries the PUF response from the target device and gets a measurement $R = PUF(C)$, where C is the selected challenge. For the weak PUF like SRAM PUF or DRAM PUF, C would be the selected address with higher reliability. Due to the algorithm operating in the finite field $GF(2^{16})$, the PUF measurement R is partitioned into f 16-bit elements $R_1, R_2, \dots, R_f$, where $f > t + 1$. Treating the elements of R as x-coordinate values. The polynomial P is evaluated at all the elements of R to obtain a set $Q = \{(R_i, P(R_i)) | i = 1, 2, \dots, f\}$. To reconstruct the polynomial P , only $t + 1$ elements in R need to be stable. Therefore, our proposal has the so-called error-tolerant feature, which ensures that the protected message can reproducibly be generated from the noise PUF. As the difference between f and t getting larger, the error-tolerant feature of our proposal will be improved. In this step, the secret data is merged in the hardware-based unique fingerprint.

The adversary may have access to the vault. Therefore, set Q need to be protected to prevent attackers reconstruct the polynomial. Some random chaff points can be added in the vault to improve the difficulty of brute-force polynomial reconstruction. The chaff points should

be randomly distributed in the vault. Thus, it should be generated by a random number generator. Prior works have shown that many PUFs can be used as a true random number generator. Therefore, a natural method to generate chaff points would be to extract some random data from the intrinsic PUF. One good choice is the DRAM PUF-based random number generator [11], which is accessible at the run-time of the commodity device without any hardware modifications. Furthermore, for the chaff points set $CF = \{(a_j, b_j) | j = 1, 2, \dots, g, g \gg f\}$, the number of chaff points g in the vault should be much larger than the number of points (f) in Q .

Then, the resulting vault $V = \{Q \cup CF | g \gg t\}$ is generated, where Q is the points with secret data that locked by PUF and CF is the set of chaff points generated by the PUF-based random number generator. The vault V can be stored in NVM. An adversary maybe has access to V as it is stored publicly on the device. The security of the vault V depends on the difficulty that the attacker can tell the real points Q from a large number of chaff points without any information of PUF on the device.

In the *UNLOCK* procedure, the device attempts to release the secret data M from the vault V by the fuzzy PUF.

The device queries the PUF response from the target device and gets a fresh measurement $R' = PUF(C)$, where C is the same challenge with the *LOCK* phase. Due to the noise effects, R' and R may not equal. Like the *LOCK* phase, the new PUF measurement R' is partitioned into f 16-bit elements $R'_1, R'_2, \dots, R'_f$. Based on the fuzzy PUF template R' , a fuzzy set $Q' = \{(R'_i, V(R'_i)) | i = 1, \dots, l\}$ can be selected out from vault V by matching $R'_1, R'_2, \dots, R'_f$ with those of the possible points. The noise effects may result in mismatching of some points. Therefore, $l \neq f$.

Although the PUF template suffers from the noise effects, the feature of error-tolerant of polynomial reconstruction allows the *UNLOCK* process to recover the secret data. For the points set Q' , $t + 1$ points are necessary to construct a t -th-order polynomial P' by Lagrange combinations. As there are l points in Q' , the number of possible combination is C_l^{t+1} . The polynomial P can be reconstructed when at least $t + 1$ elements in R can reproducibly be generated in R' . The polynomial P' that satisfied the CRC-16 checking is the right polynomial. The coefficients of polynomial P' is the secret data M that the device stored in the *LOCK* process. The other method that can replace the CRC-16 chacking is the hash algorithm. In the *LOCK* process, $hash(M)$ can be stored in the NVM and the hash value then can be used to check the right polynomial.

4. Implementation and Evaluation

Our proposal was evaluated on commodity off-the-shelf Raspberry Pi B+ devices, without any hardware modifications. The evaluation considers the varying thermal conditions in a thermal chamber.

Algorithm 1: *LOCK* process

M : k -bit message to be stored by the device
 S : n -bit codewords encoded by M
 P : t -th-order polynomial
 C, R : challenge and response of PUF
 ads, ade : start and end address of DRAM or SRAM region used as PUF
 Q : points set with f points
 CF : points set with g points

- 1: Input secret message M ;
- 2: Encode message M by an error detecting code:
 $S = ENC(M)$;
- 3: $S = \{S_0, \dots, S_t\}$;
- 4: Construct $P(X)$: $P(x) = S_t \cdot x^t + S_{t-1} \cdot x^{t-1} + \dots + S_0$;
- 5: Query PUF: $R = PUF(C, ads, ade)$;
- 6: $R = R_1, R_2, \dots, R_f$;
- 7: Evaluate the points set $Q = \{(R_i, P(R_i)) | i = 1, 2, \dots, f\}$;
- 8: Generate the chaff points set
 $CF = \{(a_j, b_j) | j = 1, 2, \dots, g, g \gg f\}$;
- 9: Output $V = \{Q \cup CF | g \gg f\}$;

4.1. Implementation in Raspberry Pi B+

The decay-based DRAM PUFs were implemented and evaluated on three commodity off-the-shelf Raspberry Pi B+ devices, labeled *Pi1*, *Pi2*, and *Pi3* without any hardware modifications.

The decay-based DRAM PUFs implementation is based on the paper [12] [13], which modified the open-source Raspberry Pi B+ firmware [14] to allow for controlling the DRAM refresh from the software.

In our proposal, the PUFs template should have high randomness to make sure the real points are randomly distributed. Therefore, the PUF template extraction method should be designed to avoid the problem of high bias in decay-based DRAM PUF.

A solution was proposed by Wenjie et al. [6]. Their approach is to identify a similar number of stable fast and slow cells to avoid the high bias problem, as there are many more slow cells than fast cells in a DRAM module. In the process, the stable cells and slow cells should be randomly selected and form the key in random order.

In this paper, we follow a simpler approach similar to the method in [6]. In DRAM PUF, although the number of bit flips is very small, the distribution of the unstable cells are random. Our test results show that there are a similar number of rows that have unstable cells and the rows without unstable cell. Therefore, we identify the rows that have bit flips as a one-bit response "1" and the rows that all the cells are stable as "0". For Raspberry Pi B+, the number of rows in the 512MB DRAM is 131072. The PUF template used in the vault can be randomly selected from all the DRAM rows with high entropy.

4.2. Experimental Setup

Figure 2 presents a schematic of the experimental setup used to test the reliability of our proposal. The setup in-

Algorithm 2: UNLOCK process

m : The maximum number of reconstruction process;
 V : Points set with $f + g$ points.
 M : k -bit message to be stored by the device
 P' : t -th-order polynomial
 C, R' : challenge and response of PUF
 ads, ade : start and end address of DRAM or SRAM region used as PUF
 Q' : points set with l points
 $CHECK$: Checking process of the error detection code;

```

for  $j = 1; j < m; j = j + 1$  do
  1: Input Vault  $V$ ;
  2: Query PUF:  $R' = PUF(C, ads, ade)$ ;
  3:  $R' = \{R'_1, R'_2, \dots, R'_f\}$ ;
  4: Matching all the elements of  $R'$  in  $V$ :
      $Q' = \{(R'_i, V(R'_i)) | i = 1, \dots, l\}$ ;
  5: Polynomial reconstruction:
     for  $i = 0; i < \binom{l}{t+1}; i = i + 1$  do
       Select  $t + 1$  points from  $Q'$ ;
        $P'(x) = S'_t \cdot x^t + S'_{t-1} \cdot x^{t-1} + \dots + S'_0$ 
       if  $CHECK(\{S'_t, \dots, S'_0\}) = 0$  then
         Accept the polynomial;
          $M = DEC(\{S'_t, \dots, S'_0\})$ ;
         Break the for loop;
       else
         Continue the for loop;
       end
     end
  if any polynomial is accepted then
    Break the for loop;
  else
    Continue the for loop;
  end
end
Output  $M$  or reconstruction faild message;
  
```

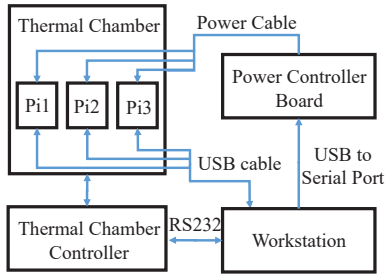


Figure 2. The experiment setup. The workstation can control the remote thermal chamber and the power controller board which is based on an Rpi board, and gather data for the decay-based DRAM PUFs.

cludes the following equipments: A thermal chamber with a thermal chamber controller, which can be controlled by the workstation through RS232; Inside the thermal chamber, three Raspberry Pi B+ boards are placed for experiments under controlled temperature. The Raspberry Pi B+ boards communicate with the workstation via the serial ports. The workstation further runs Python scripts for controlling the thermal chamber and the Power controller board to provide stable and controllable temperature and power supply.

As shown in paper [6], the temperature variations have

a much greater impact on the decay rate of decay-based DRAM PUF than manufacturing variations. For example, a higher temperature may improve the decay rate of DRAM PUF. Under a certain decay time, DRAM PUFs may generate more unexpected bit flips during system operation, introducing errors in PUF-based key generation or authentication system. Fortunately, decay-based DRAM PUFs has a clear correlation between temperature and decay time. The DRAM PUF at temperature T' for decay time t' have the same response with decay time t at temperature T . For one of our Raspberry Pi B+ board (*Pi1*), the relationship is:

$$t'_{T'} = t * e^{-0.074084*(T'-T)} \quad (1)$$

Based on the Equation 1, we query DRAM PUF under different temperature and decay time: $(t = 15^\circ C, T = 93s), (t = 20^\circ C, T = 65.5s), (t = 25^\circ C, T = 46s), (t = 30^\circ C, T = 32.5s), (t = 34^\circ C, T = 24.6s), (t = 40^\circ C, T = 15.4s)$ for *Pi1*, *Pi2*, and *Pi3*. For each test conditions, we do 20 times PUFs query. Considering the heat up process of the chip itself, we just take the last 15 times DRAM PUFs data into consideration.

4.3. Reliability Evaluation

The reliability of our proposal depends on the robust of PUF template. Under a certain PUF template's reliability, the error-tolerant parameters (t and f) can be selected to meet the reliability requirements. To understand the reliability of our proposal, we evaluated the DRAM PUF template reliability using the thermal chamber.

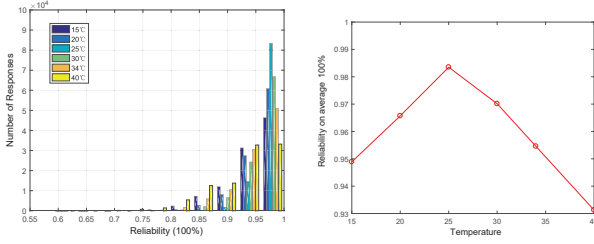
Given the large amount of memory present, we measured multiple DRAM rows, totaling 16MB (4096 rows) on each device. The rows were spread across the whole chip. To mitigating the bias of decay-based DRAM PUF, as aforementioned, each row forms one-bit response. In our proposal, the PUF template is consists of f 16-bit elements. Some elements should be remain stable (100% reliability for all the 16-bit PUF responses) when used in-the-field because it will be used to reconstruct the polynomial.

As shown in Figure 3, 10×10^4 combinations of 16-bit elements are randomly selected from all the 4096 DRAM rows. Each element was measured at six temperatures $t = 15^\circ C, 20^\circ C, 25^\circ C, 30^\circ C, 34^\circ C, 40^\circ C$. We take one of the 15 tests under $25^\circ C$ as the k in Equation 2. The reliability evaluation was done using Formula 2 shown below, where HD is the Hamming Distance.

$$reliability = \frac{HD(k, k')}{n} \times 100\% \quad (2)$$

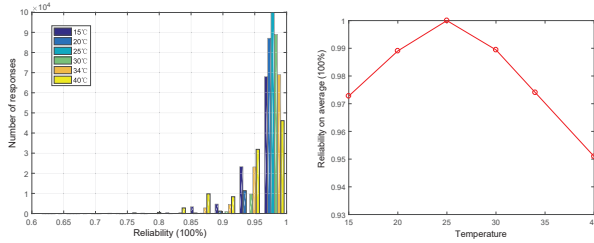
The reliability of the selected 16-bit elements fluctuates. As shown in Figure 3a, there are just less than 70% of the randomly selected PUF elements distributed under 100% reliability at $30^\circ C$, which is the best test results. For all the tests, the average reliability fluctuates from 93% to 98.5%. The low-reliability means we need higher error-tolerant feature in the polynomial reconstruction process.

To improve the reliability of DRAM PUF template, we improve the noise and temperature resilience by selecting only those rows that are unlikely to change their response when environmental conditions change. As shown in Figure 4a, nearly 90% of the randomly selected elements distributed under 100% reliability. As shown in Figure 4b the average reliability of PUF template was improved to the range of 95% to 1%. Based on the test results, the parameters can be selected to make sure the reliability of the *UNLOCK* process. For example, under 30°C, the error-tolerant of polynomial should be more than 10%. For example, for the $t = 10$, f should larger than 12.



(a) The reliability distributions of DRAM PUF template under different temperature. (b) The reliability of DRAM PUF template on average under different temperature.

Figure 3. The reliability of DRAM PUF template under 15°C, 20°C, 25°C, 30°C, 34°C, 40°C.



(a) The reliability distributions of DRAM PUF template with selected rows under different temperature. (b) The reliability of DRAM PUF template with selected rows on average under different temperature.

Figure 4. The reliability of DRAM PUF template with selected rows under 15°C, 20°C, 25°C, 30°C, 34°C, 40°C.

5. Security Analysis

In our proposal, the adversary has access to the vault as it is stored publicly on the device. The security of vault relies mainly on the number of chaff points and real points. The security of vault can be evaluated using Equation 5, where l is the number of possible real points selected in the *UNLOCK* phase, t is the degree of the polynomial, g is the number of chaff points, and f is the number of real points formed in the *LOCK* process. H_∞ roughly corresponds to the same level of difficulty in brute-force attack a full-entropy password.

$$H_\infty = (Q|V) = -\log\left(\frac{C_l^{t+1}}{C_{g+f}^{t+1}}\right) \quad (3)$$

The Formula 5 shows that the entropy of the vault improves as the number of chaff points increasing because the adversary must guess more times in the brute-force attack. Under certain f and g , the vault with larger t has better security, because the attacker has to pick out more real points from the vault to reconstruct the polynomial. However, the security of vault decreases as the number of real points increasing, although the vault has relatively higher reliability. Therefore, the vault is a trade-off between security and reliability. Furthermore, paper [15] demonstrate that the minutiae descriptors can be used to encrypt the ordinate value of Q , which improves the security of vault further.

In the case of brute-force attack, the probabilities of a successful attack can be evaluated by H_∞ aforementioned. Besides this attack, there are another three attack methods should be concerned: Correlation attack [16], Blended Substitution Attack [17] and Chaff Points Distance Attack.

Correlation Attack [18] is an attack that the adversary can obtain at least two vaults that belong to the same PUF. Although these vaults are created by different secret data and different chaff points, the x-ray value of the real points should highly overlap since they are the minutiae points from same PUF [16]. Therefore, some chaff points will be filtered out and the attacker could obtain the real points easier.

In the Blended Substitution Attack, the user can lock the secret data in the vault based on the private PUF template. In parallel, the attacker injects his attacker template into the user's vault. In the unlock process, both the user and the attacker can unlock the vault in success [17].

Chaff Points Distance Attack [18]: In the lock process, the distances between chaff points and real points are greater than a controllable parameter. But the distance between real points is completely random. Therefore, if the distance of two points are less than the controllable parameter, these two points are all real points. An attacker can filter out the chaff point with this method.

In our design, each PUFs is used in one embedded device for secret data protection. Therefore, the attacker cannot obtain two vaults that belong to one PUFs.

For the Chaff Points Distance Attack, paper [18] proposed a novel chaff point placement method to handle this problem by adjusting the distance between real-to-chaff and chaff-to-chaff points. They also mentioned that using a special hash function may avoid the Correlation Attacks further.

To resist the Blended Substitution Attack, it must be assumed that an adversary has no access to the *LOCK* process and PUFs. All the steps should be operated under a security boundary like the enrollment phase in the fuzzy extractor.

Furthermore, Paper [19] demonstrates an unlinkable Fuzzy Vault for multiple fingerprints, providing an acceptable security level against offline attack. This method is also suitable for our proposal. There are existing several components in one embedded system which can be used as PUF to protect the secret data, e.g., SRAM PUF, DRAM

PUF, and Nandflash PUF. Further, each SRAM PUF, DRAM PUF, and Nandflash PUF array can be seen as several PUF templates to operate the data protection process.

Paper [20] proposed that embed some fake polynomials in the vault can increase the security of secret polynomial further. Also, some improved Fuzzy Vault scheme [21] [22] can be used to validation the scheme.

6. Conclusion

In this work, we presented that the important data can be protected by the fuzzy PUF template without any error correction algorithm. We evaluated possibilities for data protection based on the decay-based DRAM PUFs on the off-the-shelf devices without any hardware modifications. The evaluation of the proposal showed its feasibility and high reliability. We further presented the security threats of our proposal as well as the possible solutions based on the feature of PUFs. This paper gives a new train of thought about PUFs-based data protection scheme.

Acknowledgments

Paul Chin and Yuan Cao have the same contribution to this work.

This work was supported in part by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, and by Zhejiang Key R&D Plan(Grant No. 2019C03133).

References

- [1] J. Delvaux, D. Gu, D. Schellekens, and I. Verbauwhede, "Helper Data Algorithms for PUF-Based Key Generation: Overview and Analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 34, no. 6, pp. 889–902, 2015.
- [2] S. Dries, "Design and analysis of trusted computing platforms," Ph.D. dissertation, Catholic University of Leuven, 2013.
- [3] P. Tuyls, B. Skoric, and T. Kevenaar, *Security with Noisy Data: Private Biometrics, Secure Key Storage and Anti-Counterfeiting*. Berlin, Heidelberg: Springer-Verlag, 2007.
- [4] J. Ari and S. Madhu, "A fuzzy vault scheme," *Designs, Codes and Cryptography*, vol. 38, no. 2, pp. 237–257, Feb 2006.
- [5] D. Y. O. R. R. L. and S. A., "Fuzzy extractors: How to generate strong keys from biometrics and other noisy data," *SIAMJ. Comput*, vol. 38, no. 1, pp. 97–139, 2008.
- [6] A. Schaller, W. Xiong, N. A. Anagnostopoulos, M. U. Saleem, S. Gabmeyer, B. Skoric, S. Katzenbeisser, and J. Szefer, "Decay-Based DRAM PUFs in Commodity Devices," *IEEE Transactions on Dependable and Secure Computing*, vol. 16, no. 3, pp. 1–14, 2019.
- [7] W. Xiong, A. Schaller, N. A. Anagnostopoulos, M. U. Saleem, S. Gabmeyer, S. Katzenbeisser, and J. Szefer, "Run-time Accessible DRAM PUFs in Commodity Devices," in *Proceedings of the Conference on Cryptographic Hardware and Embedded Systems*, ser. CHES, August 2016.
- [8] M. S. Hashemian, B. Singh, F. Wolff, S. Clay, and C. Papachristou, "A robust authentication methodology using physical unclonable functions in dram arrays," in *Design, Automation Test in Europe Conference Exhibition*, ser. DATE, March 2015, pp. 647–652.
- [9] J. S. Kim, M. Patel, H. Hassan, and O. Mutlu, "The dram latency puf: Quickly evaluating physical unclonable functions by exploiting the latency-reliability tradeoff in modern commodity dram devices," in *2018 IEEE International Symposium on High Performance Computer Architecture*, ser. HPCA, Feb 2018, pp. 194–207.
- [10] J. S. Sobolewski, "Cyclic redundancy check," in *Encyclopedia of Computer Science*. Chichester, UK: John Wiley and Sons Ltd., pp. 476–479.
- [11] C. Keller, F. Grkaynak, H. Kaeslin, and N. Felber, "Dynamic memory-based physically unclonable function for the generation of unique identifiers and true random numbers," in *IEEE International Symposium on Circuits and Systems*, ser. ISCAS, June 2014, pp. 2740–2743.
- [12] S. Chen, W. Xiong, Y. Xu, B. Li, and J. Szefer, "Thermal covert channels leveraging package-on-package dram," in *Proceedings of the International Conference on Trust, Security and Privacy in Computing and Communications*, ser. TrustCom, August 2019.
- [13] S. Chen, B. Li, and Y. Cao, "Intrinsic Physical Unclonable Function (PUF) Sensors in Commodity Devices," *Sensors*, vol. 19, no. 11, p. 2428, 2019.
- [14] B. Alex, R. Alyssa, and B. Kristina, "rpi-open-firmware," <https://github.com/christinaa/rpi-open-firmware>.
- [15] Feng Hao, R. Anderson, and J. Daugman, "Combining crypto with biometrics effectively," *IEEE Transactions on Computers*, vol. 55, no. 9, pp. 1081–1088, Sep. 2006.
- [16] A. Kholmatov and B. Yanikoglu, "Realization of correlation attack against the fuzzy vault scheme," *The International Society for Optics and Photonics*, vol. 6819, 2008.
- [17] S. Hong, W. Jeon, S. Kim, D. Won, and C. Park, "The vulnerabilities analysis of fuzzy vault using password," in *International Conference on Future Generation Communication and Networking*, vol. 3, Dec 2008, pp. 76–83.
- [18] C.rencik, C. Orencik, T. B. Pedersen, E. Sava, E. Savas, M. Keskinz, and M. Keskinoz, "Improved fuzzy vault scheme for fingerprint verification," *Springer-Verlag*, 2008.
- [19] K. Nandakumar, A. K. Jain, and S. Pankanti, "Fingerprint-based fuzzy vault: Implementation and performance," *IEEE Transactions on Information Forensics and Security*, vol. 2, no. 4, pp. 744–757, Dec 2007.
- [20] C. rencik, C. Orencik, T. B. Pedersen, E. Sava, E. Savas, M. Keskinz, and M. Keskinoz, "Securing fuzzy vault schemes through biometric hashing," *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 18, no. 4, pp. 515–539, 2010.
- [21] N. Minh, Tan, T. Quang, Hai, and D. Tran, Khanh, "Enhance fuzzy vault security using nonrandom chaff point generator," *Information Processing Letters*, vol. 116, no. 1, pp. 53–64, 2016.
- [22] L. Leng and A. B. J. Teoh, "Alignment-free row-co-occurrence cancelable palmprint fuzzy vault," *Pattern Recognition*, vol. 48, no. 7, pp. 2290–2303, 2015.